6회차 · CDC / Iceberg 프로젝트 스터디

바뀐 것만 고치고도 전체 계산과 같은 답으로

모델 계약 · 컬럼 리니지 · 증분 범위 · 소급 정정과 재시도

약속한 열 모양을 지키고 영향받은 키와 날짜만 골라 Gold를 갱신합니다

8회차 로드맵과 5회차 기준 구현

수집 · 적재 (1~3회차)

오늘 집중 구간: 계약 · 리니지 · 증분 Gold

PostgreSQL
원천 DB
주문·결제·환불

→

Debezium + Kafka
CDC 이벤트
토픽별 보관

→

Flink → Bronze
토픽별 테이블
원문 append

→

Silver 현재 상태
LSN MERGE
is_deleted 보존

→

Silver core
dim · fct 3개
열 계약 적용

→

Gold
주문·일별 마트
영향 범위
MERGE

→

정산·카탈로그
지표 조회
컬럼 리니지

5회차 기준 → 7·8회차 연결

모델 5개 전체 교체·계약 + 테스트 15개(PASS=20) → 7회차 파일·스냅샷, 8회차 재시도

오늘의 격리 실험판

공통 테이블은 그대로 두고 lake.lesson06에서 s1~s6 증분·복구 실험

계산 규칙은 5회차와 같고, 갱신 범위만 전체에서 영향 키·날짜로 좁힙니다

오프닝 데모: 환불 한 건이 바뀔 때 어디만 고칠까

직전 Gold (s2)

o1(11-01): 결제 100,000 · 순 80,000
o2(11-02): 결제 0 · 순 0
o3(11-02): 결제 50,000 · 순 50,000

새 변경 (s3)

refunds의 r1만 20,000 → 30,000
payment_id = p1 (order_id 없음)
orders · payments 변경 없음

부분 갱신과 대조

영향 키·날짜만 골라 MERGE
Explorer로 전후 행·계획 확인
전체 계산과 양방향 EXCEPT 0/0

```
-- labs/session06/sql/03-order-merge.sql 핵심
MERGE INTO {gold_order} t USING (
  SELECT k.order_id, n.order_date, n.paid, n.refunded, n.net_paid,
         n.order_id IS NOT NULL present
  FROM {affected_keys} k LEFT JOIN {full_order} n ON k.order_id = n.order_id
) s ON t.order_id = s.order_id
WHEN MATCHED AND NOT s.present THEN DELETE
WHEN MATCHED THEN UPDATE SET t.refunded = s.refunded, t.net_paid = s.net_paid ...
```

실측(11/2, s3) · 환불 한 건 수정 시 영향 키·날짜만 고쳐 전체 계산과 0/0 일치

왜 데이터 테스트만으로는 부족할까

지키는 대상	어디에 적나	확인 시점과 실무 의미
열 이름·타입 (모델 계약)	_*__models.yml contract	테이블 생성 전 · 미합의 스키마 변경 차단
데이터 내용 (키·관계·금액)	YAML tests · tests/*.sql (15개)	테이블 생성 뒤 · 하류 모델은 SKIP
설명·열 리니지·계약 현황	OpenMetadata 2.0.5 (선택)	컴파일 SQL 계보와 검증 결과를 한 화면에

실측(11/1) · Silver core 3개 + Gold 2개 모두 contract 적용 후 PASS=20

모델 계약 위반 예측과 Spark의 제약 한계

위반 시연 조건

복사본 YAML에서만 타입 변경
fct_payment.amount: string
원천 SQL 결과는 BIGINT

Spark 제약의 한계

dbt-spark 1.9.3 코드 확인
not_null 등 constraints는
모두 NOT_ENFORCED

우리 저장소의 분담

열 이름·타입 → contract
키 유일성·결측·정산 차액 →
data tests 15개

```
# 1) contract-failure-demo.sh: 복사본 YAML만 바꿔 dbt run 실행
models:
  - name: fct_payment
    config: {contract: {enforced: true}}
    columns: [{name: amount, data_type: string}]   # 원래 bigint
-- 2) 정상 계약 빌드 시 컴파일된 DDL (silver.dim_order)
create or replace table silver.dim_order using iceberg as
select order_id, ordered_at, order_date, status from (...)
```

실측(11/1) + 코드 · 계약은 교체 전 차단, 값 제약은 테스트 15개로 지킵니다

OpenMetadata에서 리니지 보기

출발 열	중간 모델·열	도착 Gold 마트·열 (11/1 실측)
silver.orders.ordered_at	dim_order.order_date	주문 · 일별 마트 order_date
silver.fct_payment.status	fct_refund.payment_status	주문 마트 환불 집계 조건
silver.fct_payment.amount	주문 마트 paid · net_paid	일별 마트 paid · net_paid
silver.fct_refund.amount	주문 마트 refunded · net_paid	일별 마트 refunded · net_paid

실측(11/1) · 메타데이터 18건·dbt 24건·계약 5개 Success · 열 계보 시각화

리니지에서 증분으로: amount 하나를 정정하면?

컬럼 리니지 (열 단위)

fct_payment · fct_refund의
amount가 바뀌면 두 마트의
금액 열들이 영향을 받음

증분 처리 (행 단위)

결제 p1 한 건만 정정됐을 때
두 마트 전체를 다시 쓸까?
어느 행만 다시 계산할까?

열의 계보에 키의 연결 경로를 더해야 다시 계산할 행 범위가 나옵니다

① 언제 전체 재계산을 증분으로 바꿀까

전체 교체 (5회차)

매번 전체 읽기 · 전체 쓰기
로직이 단순하고 복구가 쉬움
감당되는 동안은 기본 선택

도입 전 판단 질문 (설계)

전체 시간 vs 목표 주기(30분 가정)
주기당 실제 변경 키·날짜 비율
범위 계산·MERGE 비용 비교

전환과 대조 원칙

전체 재계산으로 정답 기준 확정
증분 결과를 기준과 대조
측정 전 속도 향상 단정 금지

변경 비율과 목표 주기(30분 가정), 범위 계산 비용을 먼저 따져 전환합니다

증분 완료 조건과 정기 전체 대조

완료 조건 (6회차 기준)	놓치면 생기는 일	실험 단계
진행 위치 유보 · 재시도 일치	실패 구간 유실 · 두 마트 불일치	s6 실패·재시도
영향 주문 전체 재합산	차액만 더하다 중복·부호 꼬임	s1 · s2 · s3
삭제·철회 키 포함	철회된 금액·삭제 행이 잔류	s4 환불 철회
이전·새 날짜 모두 갱신	옛 날짜에 유령 합계가 남음	s5 · s6 정정

매 단계 EXCEPT 0/0 검증은 운영의 정기 전체 대조(reconciliation)에 해당합니다

② 변경을 어느 테이블에서 읽을까

후보 A: Silver core

dim_order · fct_payment 등
매번 CREATE OR REPLACE
이미 where not is_deleted

후보 B: Silver 현재 상태

orders · payments · refunds
바뀐 키만 LSN 조건 MERGE
is_deleted = true 보존

전체 교체 테이블의 스냅샷 차이는 업무 변경이 아니므로 현재 상태에서 읽습니다

create_changelog_view와 COW carry-over 확인

구간과 전후 행 묶기

start 제외 · end 포함 구간
identifier_columns 지정 시
UPDATE_BEFORE / AFTER

s5 파일 재작성

s1(o1·o2 한 파일) · s2(o3 추가)
s5에서 o1 날짜만 수정 시
overwrite (added 1 · deleted 1)

carry-over 자동 제거

COW 재작성 때 값 변화 없이
새 파일로 따라온 이웃 행을
기본 동작으로 제거

```
CALL lake.system.create_changelog_view(
  table => 'lesson06.s06_demo_s5_orders',
  options => map('end-snapshot-id', '5247711452044697834',
                 'start-snapshot-id', '2118278423232506279'),
  changelog_view => 's06_..._changes_orders',
  identifier_columns => array('order_id'),
  compute_updates => true)
```

실측(11/2, s5) · overwrite 스냅샷에서 carry-over를 걸러 변경 키만 남깁니다

③ 영향 키 전파와 주문 전체 재합산

orders · payments

order_id가 바로 있음
그대로 영향 키에 추가

→

refunds → payments

order_id가 없음
input_payments와 조인해
order_id로 전파

→

차액 대신 전체 재합산

영향 키만 좁히고 금액은
유효 결제·환불 전체 재합산
재실행해도 같은 금액 보장

```
CREATE OR REPLACE TEMP VIEW {affected_keys} AS
SELECT order_id FROM {changes_orders}
UNION SELECT order_id FROM {changes_payments}
UNION SELECT p.order_id
    FROM {changes_refunds} r
    JOIN {input_payments} p ON r.payment_id = p.payment_id
```

변경 감지로 키만 좁히고, 선택된 주문 금액은 전체 계산 식으로 다시 합산합니다

활동: 내 도메인 키 전파 경로 한 줄 쓰기

항목	채팅에 적을 내용과 예시
공통 예제	refunds(payment_id) → payments(order_id) → 주문 마트(order_id)
내 도메인 경로	하위 테이블(조인 키) → 중간 테이블 → Gold 마트(기준 키)
삭제·철회 확인	하위 행 삭제 시 어느 현재 상태 테이블에서 연결 키를 찾을지

내 프로젝트의 하위 변경 → 마트 기준 키 경로와 삭제 시 키 위치를 적습니다

과거 날짜가 바뀌면 어느 날짜를 다시 묶을까

소급 정정 (s5)

주문 o1의 ordered_at 정정
2026-11-01 → 2026-10-28
금액은 그대로, 날짜만 이동

새 날짜만 고치면

10-28 행은 새로 생기지만
11-01 행에 옛 100,000원이
유령처럼 그대로 남음

영향 날짜 세 원천

기존 Gold 주문의 날짜
U 새로 계산한 주문의 날짜
U UPDATE_BEFORE · DELETE

```
-- lab.py affected_dates 계산
SELECT g.order_date FROM {gold_order} g JOIN {affected_keys} k ON g.order_id = k.order_id
UNION SELECT n.order_date FROM {full_order} n JOIN {affected_keys} k ON n.order_id = k.order_id
UNION SELECT to_date(from_utc_timestamp(ordered_at, 'Asia/Seoul')) order_date
    FROM {changes_orders} WHERE _change_type IN ('UPDATE_BEFORE', 'DELETE')
```

집계 기준 열(order_date)이 바뀌면 이전 값과 새 값 양쪽을 모두 갱신합니다

④ Gold 주문 마트와 일별 마트 MERGE 읽기

```
-- [1] MERGE INTO {gold_order} (03-order-merge.sql)
SELECT k.order_id, ..., n.order_id IS NOT NULL present
FROM {affected_keys} k LEFT JOIN {full_order} n ON k.order_id = n.order_id
WHEN MATCHED AND NOT s.present THEN DELETE   WHEN MATCHED THEN UPDATE SET ...
-- [2] MERGE INTO {gold_daily} (04-daily-merge.sql)
SELECT d.order_date, count(o.order_id) order_count, sum(o.paid) paid, ...
FROM {affected_dates} d LEFT JOIN {gold_order} o ON d.order_date <=> o.order_date GROUP BY d.order_date
WHEN MATCHED AND s.order_count = 0 THEN DELETE   WHEN MATCHED THEN UPDATE SET ...
```

영향 목록을 왼쪽에 두고 LEFT JOIN해 사라진 키와 0건 날짜를 DELETE합니다

진행 위치(progress)와 계획(plan) 저장 순서

① 스냅샷 고정 · plan 저장

start ~ end 구간 고정 후
영향 keys · dates 계획을
Gold 쓰기 전에 먼저 저장

② 두 마트 순서대로 MERGE

gold_order 먼저 갱신 후
gold_daily 이어서 갱신
멈추면 plan만 남음(실패 감지)

③ 완료 후 progress · 만료

둘 다 성공한 뒤 progress 갱신
만료된 snapshot은 조회 불가
보존 기간·비상 전체 재계산

plan은 쓰기 전에, progress는 완료 후에 · 만료된 스냅샷은 이어 읽지 못합니다

⑤ s1~s5 시나리오와 철회·날짜 정정 예측

단계	입력 변경 (UTC → 서울 날짜 동일)	결과 또는 채팅 예측
s1 · s2	o1(11-01, 순 80,000)·o2(11-02, 0) 후 o3(11-02, 50,000) 추가	o1·o2 한 파일, o3 append
s3 수정	r1 환불액 20,000 → 30,000 (LSN 300)	o1 순결제 70,000 (오프닝 확인)
s4 철회	r1 is_deleted = true (LSN 400)	[예측 ①] o1의 순결제액은?
s5 정정	o1 ordered_at: 11-01 → 10-28 (LSN 500)	[예측 ②] dates와 남는 행은?

s4 환불 철회 뒤 금액과 s5 주문일 이동 뒤 남는 날짜 행을 먼저 예측해 봅니다

실측 확인: s4·s5 증분 결과와 전체 계산 대조

s4 환불 철회

refunds: ['r1'] 감지
11-01 환불 0 · 순 100,000
verify 0 / 0 통과

s5 과거 날짜 이동

keys: ['o1'] (o2 제외)
dates: ['10-28', '11-01']
11-01 행 삭제 완료

s1~s5 전체 판정

고정 기대값(expected)과
전체 계산(EXCEPT) 모두
다섯 단계 연속 일치

{"step": "carryover", "table": "orders", "raw_rows": 4, "changelog_rows": 2, "carryover_keys": ["o2"]}
{"step": "plan", "keys": ["o1"], "dates": ["2026-10-28", "2026-11-01"]}
{"step": "verify", "gold_order": {"actual_minus_full": 0, "full_minus_actual": 0, "match": true},
"gold_daily": {"actual_minus_full": 0, "full_minus_actual": 0, "match": true}}
{"step": "expected", "scenario": "s5", "match": true,
"actual": {"gold_daily": [["2026-10-28", 1, 100000, 0, 100000], ["2026-11-02", 2, 50000, 0, 50000]]}}

실측(11/2) · 수정(s3)·철회(s4)·날짜 이동(s5) 모두 전체 계산과 0/0 일치

⑥ 실패·재시도 함정 (s6): 두 마트 사이에서 멈추면?

① 1차 시도 후 중단

o1 날짜 10-28 → 10-25 이동
gold_order만 먼저 커밋되고
gold_daily 직전 강제 실패

② 같은 구간 재시도

progress는 그대로이므로
같은 스냅샷 구간을 재실행
(8회차 작업 재시도 상황)

③ 계획 미저장(naive) 시

plan 저장·UPDATE_BEFORE 없이
현재 gold_order와 새 계산만 비교하면 어떻게 될까?

반쯤 갱신된 Gold를 보고 계획을 다시 짜면 이전 날짜 정리를 놓치게 됩니다

s6 실측 비교: naive 불일치 vs 계획 재사용 복구

naive 모드 (재시도)

dates: ['2026-10-25']만
gold_daily에 10-28 잔류
actual_minus_full = 1

정상 모드 (재시도)

plan reused = true
dates: ['10-25', '10-28']
10-28 삭제 · verify 0 / 0

이 실험의 방어선

① UPDATE_BEFORE 날짜 포함
② Gold 쓰기 전 plan 저장 후
같은 구간 재시도 시 재사용

```
-- [naive 모드 재시도 실측: 불일치 발생]
{"step": "plan", "reused": false, "dates": ["2026-10-25"], "naive_dates": true}
{"step": "verify", "gold_daily": {"actual_minus_full": 1, "full_minus_actual": 0, "match": false}}

-- [정상 모드 재시도 실측: 일치]
{"step": "plan", "reused": true, "keys": ["o1"], "dates": ["2026-10-25", "2026-10-28"]}
{"step": "verify", "gold_order": {"match": true}, "gold_daily": {"match": true}, "match": true}
```

실측(11/2) · 쓰기 전 저장한 계획을 재사용해 재시도 후 전체 계산과 일치했습니다

내 프로젝트에 증분·정정·운영 기준 적용하기

설계·운영 질문	내 도메인으로 적을 것
① 도입 판단·계약·원천	전체 계산 시간 vs 주기, contract 모델, 변경 읽을 현재 상태 테이블
② 키 전파와 삭제 포착	13장에 적은 하위→마트 기준 키 경로와 삭제 시 키 보존 위치
③ 값 이동과 DELETE MERGE	이전·새 그룹을 함께 고칠 열과 빈 그룹 삭제 조건
④ 복구·만료·정기 대조	plan·progress 순서, 스냅샷 보존 기간, 정기 전체 EXCEPT 대조

집에서 demo.sh·Explorer·lab.sh로 확인한 뒤 내 도메인 기준표를 채웁니다

오늘 남길 것과 7회차 예고

거버넌스 확인

contract: 교체 전 열 모양 차단
Spark 제약은 NOT_ENFORCED
컬럼 리니지로 열 파급 추적

증분·운영 기준

현재 상태 changelog + 키 전파
옛·새 날짜 갱신 + plan 선저장
정기 EXCEPT 대조와 실패 감지

7·8회차 연결

반복 COW MERGE와 파일 재작성
progress 스냅샷 만료 주의
7회차 유지 관리 · 8회차 재시도

다음 시간: 반복 MERGE로 쌓인 작은 파일과 스냅샷 유지 관리를 측정합니다